

Entangled Enhancement in a Pipeline of Three ML Components: Hypotheses and Experimental Design

Ifty Rashedul Arefin

September 24, 2026

Abstract

Entangled enhancement is a retraining that improves one component of a machine learning (ML) system but fails to improve the whole system, because the components depend on each other. Existing models of when to retrain such a system depend on a probability that has not been measured on a real system. Earlier experiments in this study used a driving pipeline in which only the perception module was an ML component. Retraining the perception module made the perception module more accurate, and the planning error increased in 6 of 15 updates. The downstream modules were fixed rules, which cannot learn the mistakes of the perception module, and the theory of entangled enhancement describes exactly that learning. This report states two hypotheses for a pipeline in which all three modules are ML components. H1 states that an update of the perception module can reduce the performance of the prediction module and the planning module even when the perception module itself improves. H2 states that the change in the output distribution of the perception module and the prediction module, measured without ground truth, is a symptom of entangled enhancement. We describe an experimental design that trains the downstream modules in three ways to identify the cause, four pipelines built from two ML models for each downstream module, a reduced dataset of 350 scenes (4.6 GB), and the validity controls of the measurement.

1 Introduction

A machine learning (ML) system is often built as a *pipeline*, which is a set of ML components connected in a line, where each component uses the output of the preceding component. A component that comes earlier in the line is *upstream* of a component that comes later, and the later component is *downstream*. When the data seen in use changes, the components must be retrained. Retraining an upstream component can improve that component and still fail to improve the whole system, because the downstream components depend on the upstream output. Wang and Machida call this effect *entangled enhancement* and study when a system should retrain each component [1,2]. Their models need one number, c_1 , which is the probability that retraining the upstream component leaves the downstream component working. This probability has not been measured on a real system, and Wang and Machida set $c_1 = 0.75$ by assumption.

Earlier experiments in this study measured this effect in a driving pipeline built on nuScenes [3], a public driving dataset recorded in Boston and Singapore. The perception module was retrained on Singapore images after training on Boston images. In every such update the perception module became more accurate, and in 6 of 15 updates the planning error increased at the same time. Only the perception module in that pipeline was an ML component. The prediction module and the planning module were fixed rules, which cannot learn the mistakes of the perception module. The increase in planning error had a simpler cause. The retrained perception module found more objects, and a rule in the planning module that reacts to nearby objects was triggered more often. The effect was real, but the experiment did not test the cause that the theory describes.

Beyond confirming that the effect exists, the question that matters for a solution is how to find a symptom of entangled enhancement before the planning error is known. In a deployed system the ground truth is usually not available, and the accuracy of each component cannot be measured. A symptom that needs no ground truth could warn a maintainer that an update of the perception module is likely to harm the downstream components.

This report prepares the test of both questions. Its contributions are:

- two hypotheses for a pipeline in which all three modules are ML components: H1 on the existence of the effect and H2 on a symptom of the effect that needs no ground truth (Section 3);
- an experimental design with three training settings for the downstream modules, four pipelines built from different ML models, and the validity controls of the measurement (Section 4);
- a reduced dataset of 350 scenes and 4.6 GB, ten times smaller than the dataset of the earlier experiments, with the same test scenes (Section 4.4).

2 Background

Pipeline: The pipeline has three modules. The *ego vehicle* is the vehicle that the pipeline drives. The *perception module* C_1 (YOLOv11n [4]) looks at a camera image and finds the *objects* around the ego vehicle, such as other vehicles and pedestrians. The *prediction module* C_2 takes these objects and predicts where each object will move. The *planning module* C_3 takes these predictions and chooses the path that the ego vehicle will drive over the next 3 seconds, which we call the *planned path*. nuScenes is divided into *scenes*, which are driving clips of about 20 seconds, and the correct position of every object is labelled by hand. We call these labelled values the *ground truth*. An *update* is one retraining of C_1 on Singapore images. The *prediction error* is the distance between the positions predicted by C_2 and the ground truth positions 3 seconds ahead. The *planning error* is the distance between the planned path and the path the human driver actually drove, 3 seconds ahead.

Entangled enhancement: Sculley et al. [5] describe how a change to one component of an ML system changes the behaviour of the components that use the output of the changed component. Wu et al. [6] show that improving one component of a system can make the whole system worse, and name such cases self-defeating improvements. Wang and Machida [1, 2] model the retraining of a system of two components and compare retraining rules. The best rule depends on c_1 , and a different rule is recommended when $c_1 < 0.1$. A measured value of c_1 is therefore needed to choose the rule for a real system.

Two possible causes: An upstream improvement can harm the system in two ways. In the first, which we call *learned mistakes*, the downstream modules learned to work around the old mistakes of the upstream module and are disturbed when those mistakes change. In the second, which we call *uneven improvement*, the upstream module improves on average but becomes worse on the cases that matter downstream, for example distant objects or rare object types. The examples of Wang and Machida [2] show the second cause, because there the downstream component was retrained on the new upstream output and still became worse.

Detecting a change in a distribution: A *distribution* is the pattern of values that an output takes over many inputs, for example how many objects C_1 finds per image and how confident C_1 is in each object. Rabanser et al. [7] compare methods that detect a change in the input distribution of an ML model without ground truth. Two common measures of the distance between two distributions are the *Jensen–Shannon divergence*, which compares two

histograms and is 0 when the histograms are equal, and the *Wasserstein distance*, which is the average amount by which values must be moved to turn one distribution into the other. Here we apply these measures to the output of a component, to compare the same component before and after an update.

ML prediction and planning modules: Prediction modules are usually trained on the ground truth. Xu et al. [8] and Zhang et al. [9] show that the prediction error increases when the prediction module receives the output of a real perception module instead, which shows that the input seen during training matters. Zhai et al. [10] and Li et al. [11] show that a planning module that receives the speed of the ego vehicle can score well on nuScenes while the planning module ignores the objects. Such a planning module cannot be affected by C_1 , and the use of the objects must be checked before any test of H1.

3 Hypotheses

H1 (existence): In a pipeline in which all three modules are ML components, an update of the perception module can reduce the performance of the prediction module and the planning module, even when the update makes the perception module itself more accurate. This happens in each of the four pipelines described in Section 4.5, not only in one particular choice of models.

We write δ_1 for how much C_1 improves after the update, measured by how accurately C_1 finds objects, Δ_2 for how much the prediction error increases after the update, and Δ_3 for how much the planning error increases after the update. An update shows entangled enhancement when

$$\delta_1 > 0 \quad \text{and} \quad (\Delta_2 > 0 \quad \text{or} \quad \Delta_3 > 0). \quad (1)$$

H1 does not claim that every update meets condition (1). H1 claims that a clear number of updates meet the condition, in every pipeline, and that the number is larger than random variation alone produces (Section 4.3).

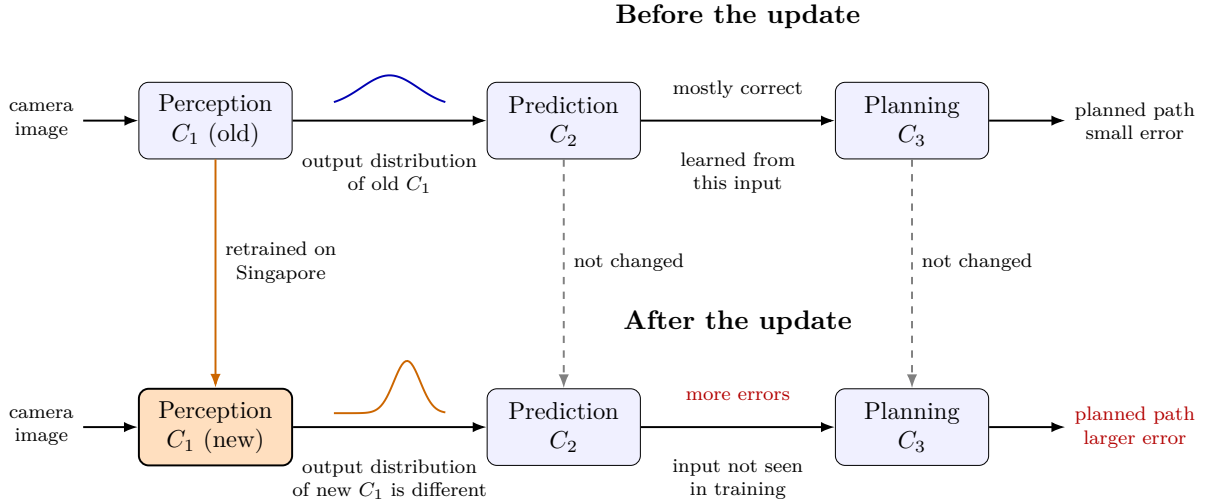


Figure 1: One update that shows entangled enhancement

Figure 1 shows one update of this kind. Only the perception module C_1 is retrained. The new C_1 finds objects more accurately, but the distribution of the output of C_1 changes, shown by the different curve. The prediction module C_2 and the planning module C_3 are not changed and were trained on the output of the old C_1 . After the update C_2 and C_3 receive an input that differs from the input seen during training, C_2 and C_3 make more errors, and the planning error increases.

H2 (symptom): The change in the output distribution of C_1 and of C_2 after an update, measured on the same test images without ground truth, is larger in the updates that meet condition (1) than in the updates that do not. The improvement δ_1 of C_1 alone does not separate the two groups of updates.

H2 is the question that leads towards a solution. If H2 holds, a maintainer can compute the change in the output distribution of C_1 and C_2 on unlabelled images before deploying an update, and treat a large change as a warning that the downstream modules need retraining as well. The different curve in Figure 1 is the quantity that H2 measures. The set of updates used to test H1 and H2 also gives the fraction of updates that meet condition (1), which is a measured value of $1 - c_1$.

4 Experimental Design

4.1 Training Settings

When the ego vehicle drives, the modules are always connected in the pipeline: the output of C_1 goes to C_2 , and the output of C_2 goes to C_3 . Training happens earlier, and each module is trained separately. During training we choose what input each module sees. The input of C_2 can be the ground truth, or the output of C_1 , including the mistakes of C_1 . We therefore train C_2 and C_3 in three ways, called *isolated training*, *old-pipeline training* and *new-pipeline training*, shown in Table 1. After training, the modules are connected into the normal pipeline in all three settings, and the same update of C_1 , from old to new, is applied. The three settings tell the two causes of Section 2 apart.

Table 1: The three ways of training C_2 and C_3 . Only the input seen during training differs. When the ego vehicle drives, the modules are connected in the same pipeline in all three settings.

Training	C_2 and C_3 learn from	Expected result if the cause is learned mistakes
Isolated	the ground truth	a better C_1 helps or changes nothing
Old-pipeline	the output of the old C_1	the planning error increases, and increases more than with isolated training
New-pipeline	the output of the new C_1	the increase seen with old-pipeline training disappears

With isolated training, C_2 and C_3 learn from the ground truth and have never seen the mistakes of any C_1 . When C_1 improves, the output of C_1 comes closer to the ground truth, and the planning error should decrease or stay the same. With old-pipeline training, C_2 and C_3 learn from the output of the old C_1 and also learn the mistakes of the old C_1 . This is the setting in which learned mistakes can act, and the planning error should increase more than with isolated training. With new-pipeline training, C_2 and C_3 are trained again on the output of the new C_1 . If the increase with old-pipeline training was caused by the old mistakes, the increase should disappear with new-pipeline training. If the planning error still increases with new-pipeline training, the cause is uneven improvement, and splitting δ_1 by object type and by distance will show where C_1 became worse. Both causes can act at the same time.

H1 is rejected if the planning error and the prediction error increase in no update, in any setting or pipeline, beyond random variation. H1 is also not testable, and the experiment must be redesigned, if the ML planning module ignores the objects (Section 4.3).

4.2 Symptom Measures

To test H2 we compute, for each update, the distance between the output distribution of the old component and the output distribution of the new component on the same 2,020 test images. None of these measures uses the ground truth. For C_1 the measures are:

- the number of objects found per image and the mix of object types, compared with the Jensen–Shannon divergence;
- the confidence scores and the sizes and positions of the found objects, compared with the Wasserstein distance;
- the *disagreement rate*, which is the fraction of objects found by only one of the two versions of C_1 on the same image.

For C_2 the measures are the Wasserstein distance between the predicted movements before and after the update, and the average distance between the positions predicted before and after the update for the same object. H2 is supported if these measures are clearly higher in the updates that meet condition (1) than in the other updates, and if the measures separate the two groups better than δ_1 does. We report the *rank correlation* between each measure and Δ_3 , which is a number between -1 and 1 that is close to 1 when a higher measure always comes with a larger Δ_3 . H2 is rejected if no measure separates the two groups better than δ_1 .

4.3 Validity Controls

Unchanged modules: We measure each downstream module in two ways. In the first way the module receives the ground truth as input, and in the second way the module receives the output of the preceding module. Because C_2 and C_3 are not changed by the update, the result with the ground truth as input must stay exactly the same, digit by digit. If the result changes, the measurement is faulty.

Random variation: An increase of the planning error counts only if the increase is larger than the increase caused by retraining the *old* C_1 with nothing changed except the *random seed*, which is the starting number for the random choices made during training. The same rule decides whether the smallest increase in the earlier experiments, 0.008 m, is real.

Two planning measures: We report the planning error together with the *collision rate*, which is the fraction of scenes in which the planned path hits an object. The earlier experiments showed that the two measures can move in opposite directions.

Use of the objects: Before any training setting is run, we remove the objects from the input of the ML planning module and check that the planned path changes. A planning module that ignores the objects cannot be affected by C_1 [10, 11].

4.4 Dataset

The earlier experiments loaded all 850 scenes of nuScenes, a 45 GB dataset, and used 559 of these scenes. This size made each experiment too slow and too large for the available computers. We built a reduced dataset of 350 scenes that keeps only the full-size images of the front camera and the label files, with a size of 4.6 GB. Table 2 compares the two datasets. Boston is split into two separate groups of scenes, one to train C_1 and one to train C_2 and C_3 , because a perception module is much more accurate on the scenes used to train that perception module [8], and C_2 and C_3 must learn from the realistic mistakes of C_1 . The 50 test scenes are the same as in the earlier experiments, which keeps the new results comparable with the old results. A first run will retrain the old C_1 with two random seeds to show whether 350 scenes are enough. If 350 scenes are not enough, we will add scenes.

Table 2: The dataset of the earlier experiments and the reduced dataset. Every scene gives about 40 front-camera images. No image is missing in the reduced dataset.

Used for	Earlier experiments	Reduced dataset
Training the old C_1 (Boston)	280 scenes	100 scenes, 4,008 images
Training C_2 and C_3 (Boston)	none, fixed rules	100 scenes, 4,046 images
Updating C_1 (Singapore)	229 scenes	100 scenes, 3,997 images
Testing (Singapore)	50 scenes	the same 50 scenes, 2,020 images
Scenes used	559	350
Size of the dataset	45 GB	4.6 GB

4.5 ML Models and Pipelines

Entangled enhancement is expected to occur in every pipeline, not only in one choice of models. We therefore select two published ML models for each downstream module and combine the models into four pipelines, shown in Table 3. All four models have public code. For C_2 , the first model is AutoBot-Ego [12], a small neural network with about 1.5 million learned numbers that was trained on all of nuScenes in about 3 hours on one graphics card. AutoBot-Ego takes the positions of the objects found in the last few images and predicts where each object will move, and can learn by itself which found objects are the same object in different images [13]. The second model is Social-LSTM [14], an older and simpler network that reads the past positions of each object one step at a time. Social-LSTM needs the same object to be matched across images first, and a second small neural network learns the matching from the object identities labelled in nuScenes. For C_3 , the first model is PlanT [15], a small neural network that takes a list of objects, here with the positions predicted by C_2 , and outputs the planned path. The second model is AD-MLP [10], the simplest published planning model for nuScenes, to which we add the predicted positions of the nearest objects as input. Neither planning model receives the speed of the ego vehicle as input, because a planning module with this input can ignore the objects [10, 11]. Pipeline P1 is run first. The other three pipelines are run once P1 has produced a result, and a model is replaced only if the model fails to train or ignores the objects.

Table 3: The four pipelines built from two ML models for each downstream module. C_1 is YOLOv11n in every pipeline.

Pipeline	Prediction module C_2	Planning module C_3
P1	AutoBot-Ego [12]	PlanT [15]
P2	AutoBot-Ego [12]	AD-MLP [10], with object positions added
P3	Social-LSTM [14], with an ML matching network	PlanT [15]
P4	Social-LSTM [14], with an ML matching network	AD-MLP [10], with object positions added

5 Conclusion

The earlier experiments in this study showed that retraining the perception module can increase the planning error, but the downstream modules were fixed rules and could not show the cause

that the theory of entangled enhancement describes. This report states H1, that an update of the perception module can reduce the performance of the ML prediction module and the ML planning module in every pipeline tested, and H2, that the change in the output distribution of the perception module and the prediction module, measured without ground truth, is a symptom of entangled enhancement. H2 is the step towards a solution, because a symptom that needs no ground truth can be computed before an update is deployed. The three training settings identify the cause, four pipelines test whether the effect depends on the choice of models, and the validity controls keep the measurement sound. The reduced dataset is built. The next step is the first run with two random seeds of the old C_1 on the reduced dataset.

References

- [1] Z. Wang and F. Machida, “Maintaining performance of a machine learning system against imperfect retraining,” in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2024, pp. 1782–1787.
- [2] Z. Wang and F. Machida, “Exploiting the availability-continuity trade-off in imperfect retraining of machine learning systems,” in *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, 2025, pp. 406–417.
- [3] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, “nuScenes: A multimodal dataset for autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11 621–11 631.
- [4] G. Jocher and J. Qiu, “Ultralytics YOLO11,” <https://github.com/ultralytics/ultralytics>, 2024.
- [5] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems (NIPS)*, vol. 28, 2015, pp. 2503–2511.
- [6] R. Wu, C. Guo, A. Hannun, and L. van der Maaten, “Fixes that fail: Self-defeating improvements in machine-learning systems,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 11 745–11 756.
- [7] S. Rabanser, S. Günnemann, and Z. C. Lipton, “Failing loudly: An empirical study of methods for detecting dataset shift,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [8] Y. Xu, L. Chambon, É. Zablocki, M. Chen, A. Alahi, M. Cord, and P. Pérez, “Towards motion forecasting with real-world perception inputs: Are end-to-end approaches competitive?” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 18 428–18 435.
- [9] P. Zhang, L. Bai, Y. Wang, J. Fang, J. Xue, N. Zheng, and W. Ouyang, “Towards trajectory forecasting from detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 10, pp. 12 550–12 561, 2023.
- [10] J.-T. Zhai, Z. Feng, J. Du, Y. Mao, J.-J. Liu, Z. Tan, Y. Zhang, X. Ye, and J. Wang, “Rethinking the open-loop evaluation of end-to-end autonomous driving in nuScenes,” *arXiv preprint arXiv:2305.10430*, 2023.

- [11] Z. Li, Z. Yu, S. Lan, J. Li, J. Kautz, T. Lu, and J. M. Alvarez, “Is ego status all you need for open-loop end-to-end autonomous driving?” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [12] R. Girgis, F. Golemo, F. Codevilla, M. Weiss, J. A. D’Souza, S. E. Kahou, F. Heide, and C. Pal, “Latent variable sequential set transformers for joint multi-agent motion prediction,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [13] X. Weng, B. Ivanovic, K. Kitani, and M. Pavone, “Whose track is it anyway? Improving robustness to tracking errors with affinity-based trajectory prediction,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [14] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, and S. Savarese, “Social LSTM: Human trajectory prediction in crowded spaces,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 961–971.
- [15] K. Renz, K. Chitta, O.-B. Mercea, A. S. Koepke, Z. Akata, and A. Geiger, “PlanT: Explainable planning transformers via object-level representations,” in *Conference on Robot Learning (CoRL)*, ser. PMLR, vol. 205, 2022.